

a (legitimate) website unless you “Confirm your username and password immediately.” Think hard and deep before you click such links, especially with the knowledge you have gained from this article. Obviously, exercise the same level of caution on online message boards and social networking sites as well.

Recent versions of Mozilla Firefox display good security features. For example, Firefox automatically encodes `<` and `>` (into `%3C` and `%3E`, respectively) in the *document.URL* property, when the URL is not directly typed into the address bar. Therefore, it is not vulnerable to DOM-based attacks. For additional security, install browser add-ons (extensions) such as NoScript, FlashBlock, and the Netcraft toolbar.

You could also try using the Google Chrome browsers, which are released with integrated XSS protection.

If you run into a doubtful link that you still want to open, if you don't use Firefox with NoScript, you should disable JavaScript, Java (and Active X, if you're on Windows) before you click the link. Alternately, visit the website by typing its address directly into your browser.

If a link is to a URL-shortening service like ‘tiny’, ‘tinyurl’, ‘bitly’, ‘is.gd’, ‘shorturl’, ‘snipurl’ etc, be careful when clicking the link. You may even want to install a second browser for ‘untrusted’ sites; in this browser, do not sign in to any of your trusted and valuable sites, but use it to visit suspicious URLs. If there is actually an attack behind the URL, even if successful, it will probably not net the attacker any useful cookies.

For developers

The best way to check your website for vulnerabilities is to run a Web application security scan against a local copy of it. The best FOSS project available for this is Nikto, which you can get at <http://www.cirt.net/nikto2>.

The next preferred option is to properly escape all untrustworthy data, based on the HTML context (body, attribute, JavaScript, CSS, or URL) that the data will be placed into. Developers need to include this escaping in their applications. See the OWASP XSS Prevention Cheat Sheet linked in the *Resources* section for more information about data escaping techniques.

Filtering script output can also defeat XSS vulnerabilities by preventing them from being transmitted to users. When filtering dynamic content, select the set of characters that is known to be safe. Instead of trying to exclude the set of characters that might be bad, this is preferable because it's unclear whether there could be any other characters or character combinations that can be used to expose other vulnerabilities.

Check all headers, cookies, query strings, form and hidden fields, and all other parameters against tags such as `<script>`, `<object>`, `<applet>`, `<form>`, and `<embed>`. Also, do not echo any input value without validation.

Do not store plain-text or weakly encrypted passwords/contents in a cookie. Merely hashing the password with MD5 is not strong, since they are just 16 bytes in length, and hence can easily be deciphered with a brute-force method.

If possible/practicable, the cookie's authentication credentials should be associated with a source IP address. Discovering the same cookie coming from different IPs should raise a red flag.

If possible, eliminate single sign-on, and apply password re-verification to avoid session takeover attacks. An attack against a site running with single sign-on has a better chance of being executed without the user noticing.

Using free hosting sites is an essential building block in the attackers' scheme. Free hosting sites need to be more vigilant about who uses their services, and enterprises should consider blacklisting suspicious IP addresses. Also, if people host scripts like cookie fetchers, they should be monitored for illegal activities on the site.

Cookies sent over HTTP(S) cannot be accessed by script via *document.cookie* so try sending cookies over HTTPS only. Also, try using the POST method for form submissions, instead of GET.

Tools of the security trade

- Dotdefender is a Web application attack protection tool that blocks attacks that are manifested within the HTTP request logic. Works perfectly for SQL injection, cross-site scripting, and header tampering. View its documentation at <http://www.applicure.com/>.
- KeepNI immediately alerts you if there is any malfunction detected on your website. It assures that your site is fully functional all the time. View its documentation at <http://www.keepni.com/>.
- Web application firewalls check for malicious input values and modification of read-only parameters and also block requests and filter out parameters. Their biggest benefit is to protect old (legacy) applications that are insecure.

Since XSS exploits the trust the user has in the website, it is equally relevant that the user remains extremely cautious. For more detailed information on XSS and more defences, don't forget to visit the resources below. We will deal with other dangerous attacks on Web applications and Apache in the next article. Meanwhile, you can send your queries and constructive feedback to abajpai75 AT yahoo DOT com. Always remember: know hacking, but no hacking! 

Resources

- [http://blog.nicircle.com/blogs/vert/miXSS Whitepaper.pdf](http://blog.nicircle.com/blogs/vert/miXSS%20Whitepaper.pdf)
- <http://www.xssed.com/> is one of the best resources for XSS information
- Syngress's XSS Attacks Cross Site Scripting Exploits and Defence by Jeremiah Grossman, Robert 'Ranake' Hansen and others, is a must-read for those who are really interested.
- [http://www.owasp.org/index.php/XSS_\(Cross_Site_Scripting\)_Prevention_Cheat_Sheet](http://www.owasp.org/index.php/XSS_(Cross_Site_Scripting)_Prevention_Cheat_Sheet)

By: Arpit Bajpai

The author is a FOSS enthusiast and loves to troubleshoot in the information security domain, while exploring new exploits and system vulnerabilities.